

	Pràctica Entregable 01 Classe per encriptar informació	IMO30FP01 REV 00 11/13
---	---	---------------------------------------

CURS:	DAM2	DATA:	Setembre-2026	Qualificació	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Encriptació i desenscriptació de dades				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				

Enunciat:

Formeu part de l'equip de desenvolupament d'una empresa que necessita **enviar petits missatges protegits entre diferents aplicacions**.

L'empresa no vol que el programa principal conegui els detalls de com es protegeixen els missatges. Per aquest motiu, **us demana crear una classe criptogràfica independent i reutilitzable**.

La classe haurà de permetre:

MISSATGE + CLAU → Procés d'ENCRIPATACIÓ → MISSATGE ENCRIPATAT

MISSATGE ENCRIPATAT + CLAU → Procés de DESENCRIPATACIÓ → MISSATGE ORIGINAL

La vostra missió és **dissenyar el vostre propi sistema de xifrat** i implementar-lo com una classe reutilitzable.

Un exemple per entendre com crear el vostre propi sistema:

Suposem que tenim:

- Missatge: HOLA
- Clau: 3

Podem utilitzar la clau per desplaçar cada lletra tres posicions:

- $H \rightarrow K$
- $O \rightarrow R$
- $A \rightarrow D$

Per tant: HOLA → KROD Per recuperar el missatge original, fem el procés invers: KROD → HOLA.

1. Investiga el funcionament

Respon aquestes preguntes:

- Què passa si encriptem dues vegades el mateix missatge amb la mateixa clau?
- Què passa si canviem la clau? Prova HOLA + clau 3 i HOLA + clau 7.

HOLA + clau 3 → KROD

HOLA + clau 7 → OVSH

- Què passa si intentem descriptar utilitzant una clau incorrecta?

Si intentem descriptar un missatge utilitzant una clau incorrecta, no recuperarem correctament el missatge original. El resultat serà diferent del missatge que es va encriptar.

Com a mínim haurà de complir:

- El mateix missatge amb la mateixa clau ha de produir el mateix resultat.
- Una clau diferent ha de modificar el resultat.
- Si utilitzem la clau correcta, hem de poder recuperar exactament el missatge original.

Descripta(Encripta("HOLA", "123"), "123") = "HOLA"

2. Dissenya l'algoritme propi d'encriptació i descriptació

Abans de programar, heu de dissenyar el vostre sistema de xifrat.

Podeu utilitzar, per exemple:

- un sistema de substitució.
- un sistema de desplaçament.
- una combinació de diferents operacions.

La clau ha de participar obligatòriament en el procés.

Nom del vostre sistema	NOVA
Tipus de sistema	☐ Substitució ☐ Desplaçament ☒ Combinació ☐ Altres: _____
Tipus de clau	Cadena de text (String)
Com funciona l'encriptació?	

	En el sistema NOVA primer convertim el missatge i la clau en Bytes per poder treballar amb numeros. Despres, relacionem cada caracter del missatge amb un caracter de la clau. Si la clau es mes curta que el missatge, la clau es repeteix des del principi. Si la clau es mes llarga, només utilitzem els caracters necessaris. Per a cada posicio, fem una operacio XOR entre el byte del missatge i el byte corresponent de la clau. Despres del XOR, sumem al resultat el numero de la posicio que ocupa el caracter dins del missatge. Finalment, convertim els bytes resultants a Base64 per obtenir el missatge xifrat en forma de text.
Com funciona la desenscriptació?	En el sistema NOVA, primer rebem el missatge xifrat en forma de Base64 i el convertim de nou en els bytes originals. Despres, per a cada byte, restem la posicio que ocupa dins del missatge. A continuacio, fem una operacio XOR amb el mateix byte de la clau que s'ha utilitzat durant l'enscriptacio. Si la clau es mes curta que el missatge, es torna a començar des del principi de la clau. Finalment, convertim els bytes obtinguts en text i recuperem el missatge original.
Com sé que el meu sistema funciona?	Podem veure que funciona fent diferents proves, Primer, comprovem que si enscriptem el mateix missatge amb la mateixa clau, obtenim sempre el mateix resultat. Després, comprovem que si canviem la clau, el resultat de l'enscriptació també canvia.

	També comprovem que si intentem descriptar el missatge amb una clau incorrecta, no recuperem el missatge original. Finalment, fem la prova més important: encriptem un missatge i després el descriptem utilitzant la mateixa clau. Si recuperem exactament el missatge inicial, el sistema funciona correctament.
--	--

3. Demostra el teu algoritme

Abans de començar a programar, feu un exemple manual complet.

Missatge: H O L A

Clau: 1 2 3 1

Posicio: 0 1 2 3

Encriptació: H → 72 O → 79 L → 76 A → 65

Encriptació Clau: 1 → 49 2 → 50 3 → 51 1 → 49

XOR:

- H XOR 1 = 121
- O XOR 2 = 125
- L XOR 3 = 127
- A XOR 1 = 112

Sumem la posicio al resultat:

- 121 + 0 = 121
- 125 + 1 = 126
- 127 + 2 = 129
- 112 + 3 = 115

Resultat de XOR + Posicio: 121 126 129 115

Covertim a Base64:

121 126 129 115 → eX6Bcw==

Resultat: eX6Bcw==

Igualment, descripta el resultat i demostra que recuperes HOLA.

Convertim el missatge xifrat de Base64 a bytes

eX6Bcw== → 121 126 129 115

Restem la posició de cada caràcter

Recordem les posicions:

- H $\rightarrow$ posició 0
- O $\rightarrow$ posició 1
- L $\rightarrow$ posició 2
- A $\rightarrow$ posició 3

Ara fem:

- $121 - 0 = 121$
- $126 - 1 = 125$
- $129 - 2 = 127$
- $115 - 3 = 112$

Obtenim:

121 125 127 112

Fem l'operació XOR amb la clau

La clau és:

- 123

Per tant:

- $121 \text{ XOR } 49 = 72 \rightarrow \text{H}$
- $125 \text{ XOR } 50 = 79 \rightarrow \text{O}$
- $127 \text{ XOR } 51 = 76 \rightarrow \text{L}$
- $112 \text{ XOR } 49 = 65 \rightarrow \text{A}$

Convertim els números a caràcters

72 $\rightarrow$ H

79 $\rightarrow$ O

76 $\rightarrow$ L

65 $\rightarrow$ A

4. Implementació de la classe "Encripta" en Java

Crea una classe independent anomenada:

ClasseCriptografica

La classe haurà d'oferir, com a mínim, aquests dos mètodes:

- `Encripta(Missatge, Clau)`
- `Desencripta(Missatge, Clau)`

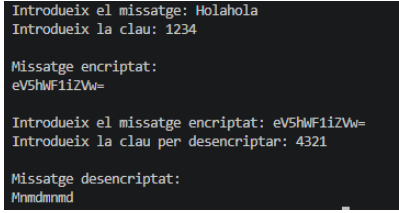
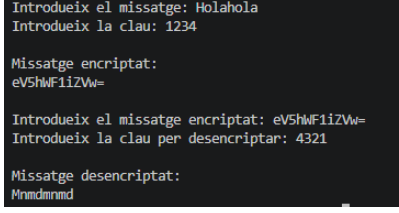
Els mètodes han de retornar el missatge resultant.

La classe criptogràfica no ha de contenir el programa principal. El programa principal serà qui utilitzarà la classe.

Programa principal → ClasseCriptografica → Encripta / Desencripta

5. Proves del sistema dissenyat d'encriptació

Test	Prova	Resultat
Test 1	Missatge curt: HOLA clau:12 <pre>Introdueix el missatge: HOLA Introdueix la clau: 12 Missatge encriptat: eX5/dg== Introdueix el missatge encriptat: eX5/dg== Introdueix la clau per desencriptar: 12 Missatge desencriptat: HOLA</pre>	☒ Correcte ☐ Incorrecte Observacions: _____
Test 2	Missatge amb espais: HOLA MÓN clau: 123 <pre>Introdueix el missatge: HOLA MÓN Introdueix la clau: 123 Missatge encriptat: eX6BcxaD5JSwIA== Introdueix el missatge encriptat: eX6BcxaD5JSwIA== Introdueix la clau per desencriptar: 123 Missatge desencriptat: HOLA MÓN PS C:\Users\Roha\Desktop\TETE\Encriptacio></pre>	☒ Correcte ☐ Incorrecte Observacions: Els accents no passen per l'encriptació.
Test 3	Missatge més llarg: almenys 30 caràcters <pre>Introdueix el missatge: Hola aquest es un missatge amb mes de 30 caracters i molt mes llarg. Introdueix la clau: 12345678 Missatge encriptat: eV5HwB1cTFrcS1eFXfILXG8jchBawmxjbnAtchRvNXR0YjVzdDsqLz16fHGAgncMc3JFkEmQjot9S5iMg10zk5CB1l0= Introdueix el missatge encriptat: eV5HwB1cTFrcS1eFXfILXG8jchBawmxjbnAtchRvNXR0YjVzdDsqLz16fHG AgncMc3JFkEmQjot9S5iMg10zk5CB1l0= Introdueix la clau per desencriptar: 12345678 Missatge desencriptat: Hola aquest es un missatge amb mes de 30 caracters i molt mes llarg.</pre>	☒ Correcte ☐ Incorrecte Observacions: _____

Test 4	Clau diferent: comprovar que el resultat canvia 	☒ Correcte ☐ Incorrecte Observacions: _____
Test 5	Encriptar i desencriptar: recuperar exactament el missatge original	☒ Correcte ☐ Incorrecte Observacions: _____
Test 6	Clau incorrecta: explicar què passa 	☒ Correcte ☐ Incorrecte Observacions: Es desencripta de manera diferent, per el que no s'aconsegueix el missatge original

6. Implementació d'un sistema de xifrat AES en Java

Fins ara heu dissenyat i implementat el vostre propi sistema de xifrat. Ara fareu un pas més: **utilitzareu AES (Advanced Encryption Standard)**, un algoritme de xifrat simètric utilitzat en sistemes reals.

En aquesta part **no haureu de crear l'algoritme AES des de zero**. Java proporciona una API de criptografia que permet utilitzar-lo.

Objectiu

Crear una classe anomenada **ClasseAES** que permeti:

MISSATGE + CLAU

↓

AES

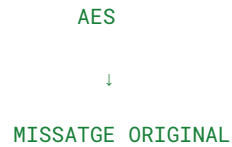
↓

MISSATGE XIFRAT

i recuperar posteriorment el missatge original:

MISSATGE XIFRAT + CLAU

↓



La classe haurà d'oferir, com a mínim, els dos mètodes següents:

```
public static String encripta(String missatge, String clau)

public static String desencripta(String missatgeXifrat, String clau)
```

6.1. Eines disponibles

Per realitzar aquesta activitat podeu utilitzar les classes i mètodes següents de Java:

Classes:

```
import javax.crypto.Cipher;

import javax.crypto.spec.SecretKeySpec;

import java.nio.charset.StandardCharsets;

import java.util.Base64;
```

Mètodes i elements que podeu consultar i utilitzar:

```
getBytes()

Cipher.getInstance()

cipher.init()

cipher.doFinal()

Base64.getEncoder()

Base64.getDecoder()

SecretKeySpec

Cipher.ENCRYPT_MODE

Cipher.DECRYPT_MODE
```

Aquestes són les eines que necessitareu per construir el vostre sistema.

No es proporcionarà el codi complet de la classe. Haureu d'investigar el funcionament de les classes i mètodes proporcionats i decidir com combinar-los per construir els processos d'encryptació i desencryptació

Trobareu un **annex explicatiu** breu al final del document.

6.2. Implementa la classe **ClasseAES**

Crea una classe independent anomenada **ClasseAES**, que tingui els mètodes:

encripta (missatge, clau)

desencripta (missatge, clau)

El mètode **encripta()** haurà de:

1. Rebre un missatge i una clau.
2. Preparar la clau perquè pugui ser utilitzada per AES.
3. Crear i configurar el sistema de xifrat.
4. Convertir el missatge al format necessari.
5. Xifrar el missatge.
6. Convertir el resultat en un format que es pugui retornar com a **String**.
7. Retornar el missatge xifrat.

El mètode **desencripta()** haurà de realitzar el procés invers:

1. Rebre el missatge xifrat i la clau.
2. Preparar la clau.
3. Crear i configurar el sistema de desxifrat.
4. Recuperar les dades xifrades.
5. Desxifrar-les.
6. Convertir el resultat novament a text.
7. Retornar el missatge original.

6.3. Programa principal

Crea un **ProgramaPrincipal** que utilitzi la classe **ClasseAES**.

Utilitza inicialment les dades següents:

Missatge: Aquest és un missatge secret.

Clau: 1234567890123456

El programa haurà de:

1. Mostrar el missatge original.
2. Encriptar-lo utilitzant AES.
3. Mostrar el missatge xifrat.
4. Desencriptar el resultat utilitzant la mateixa clau.
5. Mostrar el missatge recuperat.

El resultat final ha de demostrar que:

```
Desencripta(Encripta(missatge, clau), clau) = missatge
```

6.4. Experimenta amb el teu programa

Una vegada el programa funcioni, realitza les proves següents:

Prova 1. Clau correcta

Comprova que el missatge original es recupera correctament.

```
=== 6.3. Demostració AES ===  
Missatge original : Aquest és un missatge secret.  
Mida de la clau   : 16 bytes  
Missatge xifrat   : 3UXPE1odbnoSr4wWreU9QCSMsetZu7rBCYtwVkl6fJc=  
Missatge recuperat: Aquest és un missatge secret.
```

Prova 2. Clau diferent

Intenta desencriptar el missatge utilitzant una clau diferent.

Explica què passa.

Hem desencriptat amb la clau "6543210987654321" (també de 16 bytes, però diferent). No s'obté el missatge: `doFinal()` llança una `BadPaddingException` ("Given final block not properly padded. Such issues can arise if a bad key is used during decryption."). Amb una altra clau, AES desxifra els blocs de manera diferent i obté bytes sense sentit, de manera que el farciment de l'últim bloc no és vàlid. En casos molt poc probables el farciment podria quedar correcte per casualitat, però el text obtingut tampoc seria l'original.

Prova 3. Missatge diferent

Utilitza un altre missatge i comprova que el procés continua funcionant.

```

=== 6.3. Demostració AES ===
Missatge original : àëëóóü ç ñ ĭ;
Mida de la clau   : 16 bytes
Missatge xifrat   : h7sVHndjoH3s/umLgcRPH3up5CczF6sfXk9nb4npYC4=
Missatge recuperat: àëëóóü ç ñ ĭ;

```

Amb el missatge "Un altre missatge, més llarg, amb accents i símbols: àëëíóü ç ñ ĭ!" i la mateixa clau, es recupera exactament el text original (accents i símbols inclosos, gràcies a l'UTF-8). El resultat xifrat és més llarg perquè el missatge també ho és.

Prova 4. Clau de longitud incorrecta

Prova d'utilitzar una clau que no tingui la longitud adequada.
Observa què succeeix i explica el motiu.

```

=== 6.3. Demostració AES ===
Missatge original : Aquest és un missatge secret.
Mida de la clau   : 15 bytes
Exception in thread "main" java.lang.IllegalArgumentException: No s'ha pogut encriptar: Invalid AES key length: 15 bytes
    at ClasseAES.encripta(ClasseAES.java:69)
    at ProgramaPrincipalAES.main(ProgramaPrincipalAES.java:29)
Caused by: java.security.InvalidKeyException: Invalid AES key length: 15 bytes
    at java.base/com.sun.crypto.provider.AESCrypt.makeSessionKey(AESCrypt.java:1329)
    at java.base/com.sun.crypto.provider.AESCrypt.init(AESCrypt.java:1053)
    at java.base/com.sun.crypto.provider.ElectronicCodeBook.init(ElectronicCodeBook.java:97)
    at java.base/com.sun.crypto.provider.CipherCore.init(CipherCore.java:481)

```

En aquest cas per ser una clau més curta es incorrecta i no es pot desxifrar

6.5. Comprèn el codi

No n'hi ha prou amb aconseguir que el programa funcioni. Has de poder explicar què està fent.

Respon les preguntes següents:

1. Quina classe de Java s'encarrega de realitzar el xifrat?

`javax.crypto.Cipher`

2. Per a què serveix `SecretKeySpec`?

Per construir una clau secreta a partir d'un array de bytes i indicar que es de AES.

3. Què fa `Cipher.getInstance()`?

Crea una instància de Cipher configurada per utilitzar l'algoritme AES. Després amb `init()` indico si vull xifrar o desxifrar i li passo la clau

4. Quina diferència hi ha entre `ENCRYPT_MODE` i `DECRYPT_MODE`?

`ENCRYPT_MODE` serveix per xifrar i `DECRYPT_MODE` per desxifrar. S'utilitzen amb `init()` per indicar al Cipher quina operació ha de fer.

5. Per a què serveix `doFinal()`?

`doFinal()` finalitza el procés de xifrat o desxifrat, processa les dades que falten, gestiona el padding i retorna el resultat final.

6. Per què hem de convertir el missatge a bytes?

Perque AES treballa amb blocs de bytes de 16,24 i 32 i no amb text.

7. Per què utilitzem Base64?

Perque el resultat del xifrat son bytes qualsevols, llavors s'ha de pasar a base64 porque els converteix en un text segur.

8. Quina mida té la clau que has utilitzat?

La clau 1234567890123456 son 16 bytes.

9. Què significa que AES sigui un sistema de xifrat simètric?

Que es fa servir la mateixa clau per xifrar i desxifrar. Llavors han de coneixer les dues persones la clau.

10. Per què necessitem la mateixa clau per recuperar el missatge?

Perque l'AES fa servir la mateixa clau per xifrar tant per com desxifrar i no retornaria el missatge intencionat.

Finalment, respon:

Per què és preferible utilitzar un algoritme criptogràfic estàndard com AES en lloc de crear el nostre propi sistema de xifrat?

És preferible utilitzar AES perquè és un algoritme estàndard, públic i molt estudiat per experts en criptografia. Ha estat provat durant molts anys i se n'han analitzat possibles vulnerabilitats. En canvi, si creem el nostre propi sistema de xifrat, podem cometre errors de seguretat que facin que sigui fàcil de trencar. Per això, és més segur i fiable utilitzar algoritmes estàndard com AES.

Lliurament

Heu de lliurar en un repositori GitHub:

1. Sistema de xifrat propi

- `ClasseCriptografica.java`
- `ProgramaPrincipal.java`

2. Implementació AES

- `ClasseAES.java`
- `ProgramaPrincipalAES.java`

3. Documentació

- Document PDF amb les respostes i explicacions de la pràctica.

Cal lliurar aquesta documentació en un Repositori GitHub i que el compartiu amb la Cristina i en Carles.

Xifrat AES

El xifrat AES (Advanced Encryption Standard) va ser aprovat com estàndard de xifrat pel govern dels EUA després d'un procés de 5 anys. El xifrat va ser desenvolupat per dos criptòlegs belgues J.Daemen i V. Rijmen que el van enviar amb el nom de Rijndael.

Característiques:

- Xifrat de bloc: mida 128 bits.
- Clau de xifrat de 128,192 o 256 bits.

Actualment es considera un sistema de xifrat segur (la NSA exigeix clau de 192 o 256 bits per documents d'alta seguretat). És un algoritme ràpid, AES 256 és més ràpid que DES i per tant molt més ràpid que la solució 3DES.

Elements del xifrat:

- IV (Vector inicialització): És necessari per iniciar cada procés de xifrat. Té una mida de 16 bytes (1 bloc). El seu objectiu és evitar que dos blocs iguals tinguin el mateix criptograma.
- Clau (Key): És la principal informació per xifrar i desxifrar en els algoritmes simètrics. A la implementació per defecte de .NET s'utilitza la de 256 bytes.

Cipher mode:

ECB: xifra cada bloc per separat. Per tant, blocs idèntics donen el mateix criptograma.

CBC: s'utilitza un vector d'inicialització (IV) i cada bloc es combina amb el bloc anterior amb una operació XOR. Això assegura que blocs iguals no donin un criptograma idèntic.

Clau de xifrat:

Existeixen mètodes per obtenir la clau de forma automàtica, però això només té sentit si aquesta clau es pot enviar de forma segura. Volem obtenir la clau a partir de passphrase que introdueixi l'usuari.

La forma més còmoda per l'usuari és introduir una password o passphrase enlloc de una clau i un vector d'inicialització. Utilitzem `Rfc2898DeriveBytes` per obtenir un *hash* de la combinació dels dos textos. S'afegeix una cadena *salt* per fer més difícil l'atac per RainBow Tables. L'algoritme es pot aplicar iterativament per augmentar la complexitat.

Rúbrica:

Criteri	Què s'avalua	Punts	
1. Disseny del sistema propi	Defineix l'algoritme, la clau i explica correctament encriptació i desencriptació	1,5	
2. Implementació del sistema propi	ClasseCriptografica funciona correctament i està separada del programa principal	1,5	
3. Ús de la clau	La clau participa realment en el procés i una clau incorrecta no permet recuperar correctament el missatge	1,0	
4. Proves del sistema propi	Realitza i documenta correctament els tests proposats	1,0	
5. Implementació d'AES	Utilitza correctament l'API de Java i implementa ClasseAES amb encriptació i desencriptació	2,0	
6. Comprensió de l'API criptogràfica	Explica Cipher, SecretKeySpec, init(), doFinal(), Base64, etc.	1,0	
7. Experimentació amb AES	Realitza les proves amb diferents claus i missatges i explica els resultats	0,75	
8. Documentació i comparació	Respon les preguntes i compara el sistema propi amb AES de manera raonada	0,75	
9. Organització i lliurament	Estructura correcta del projecte, noms de classes, repositori GitHub i documentació	0,5	
TOTAL		10	